

Case Study: Software Development at Google



An analysis of Google's software engineering principles and workflows

Merve Pakcan
Software Engineering Fundamentals

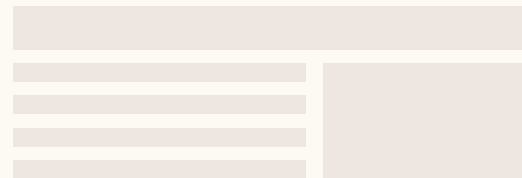




Table of contents

01

Introduction

Why Google as a case study

02

Core Themes of Google's Engineering Culture

03

Design Quality Criteria

04

Google's Architecture

05

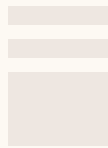
Code Review at Google

06

Design Patterns

07

Conclusion



Why Google for a Software Engineering Case Study?



Google operates one of the world's largest codebases and engineering teams.



Scale

One of the largest software engineering organizations



Experience

20+ years of continuous development



Quality

Real-world example of architecture, quality & maintainability



Relevance

Strong alignment with Software Engineering Fundamentals

Sources Used:

Software Engineering at Google: Lessons Learned from Programming Over Time (2020)

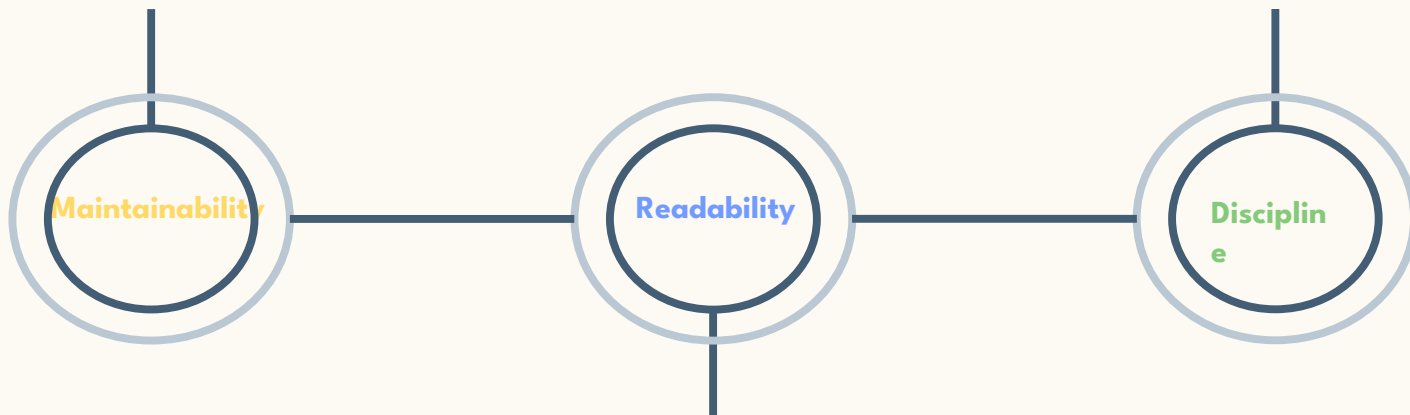
Modern Code Review: A Case Study at Google (2013)



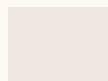
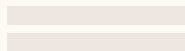
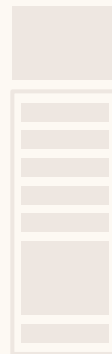
Google's Software Engineering: Core Themes

Google optimizes for **long-term maintainability**, not short-term speed.

Large codebases require **strict consistency and engineering discipline**.



Code is written for **humans first**, because thousands of engineers will interact with it.

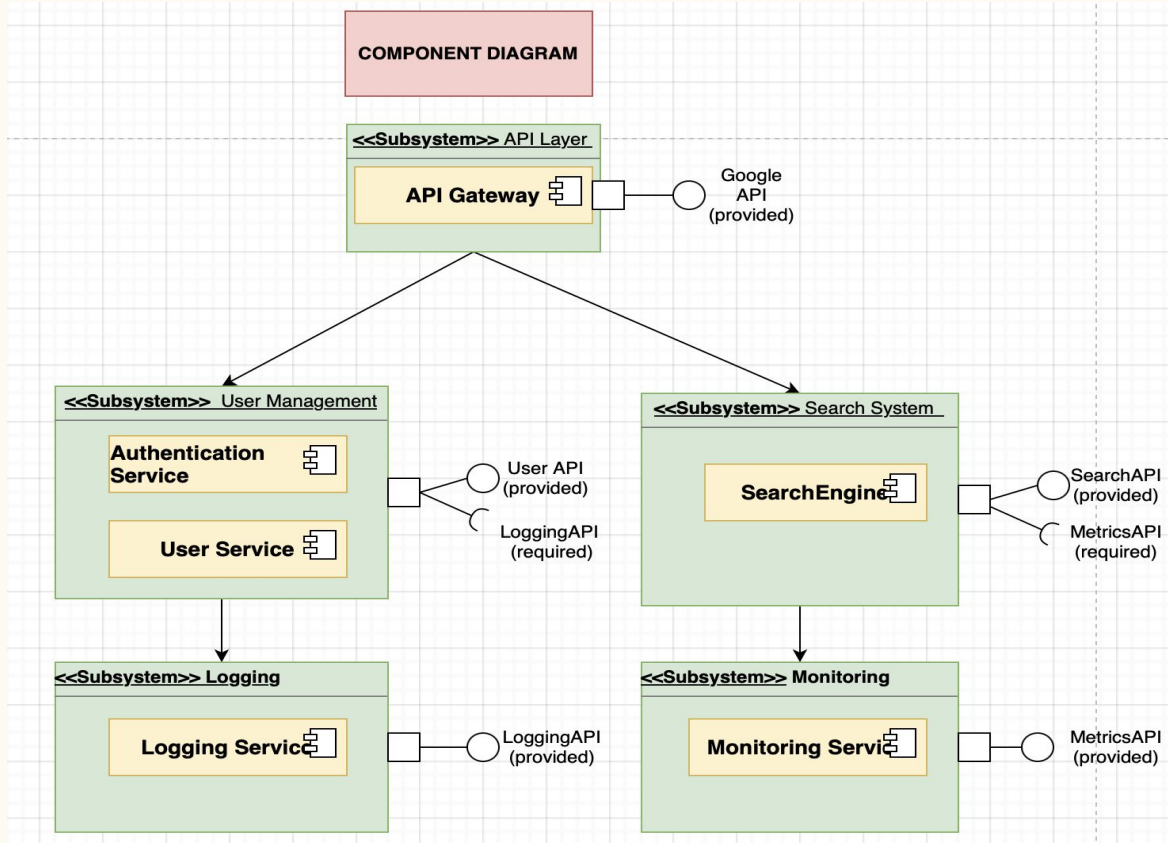




Design Quality Criteria

<u>Quality Criteria</u>	<u>How Google Applies Them</u>
 Performance 	Continuous monitoring (Google Search, Maps and YouTube)
 Reliability 	Architectural reviews (Gmail, Google Cloud and Drive)
 Maintainability 	Readability Program
 Modularity 	API boundaries, component ownership

Google's Architecture Approach





Code Review at Google

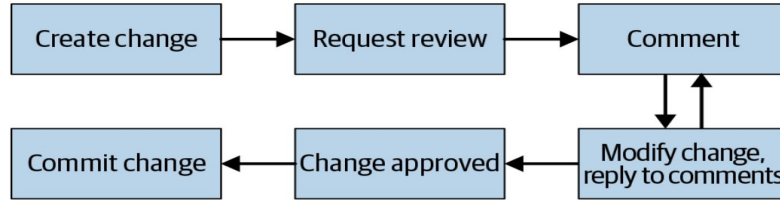
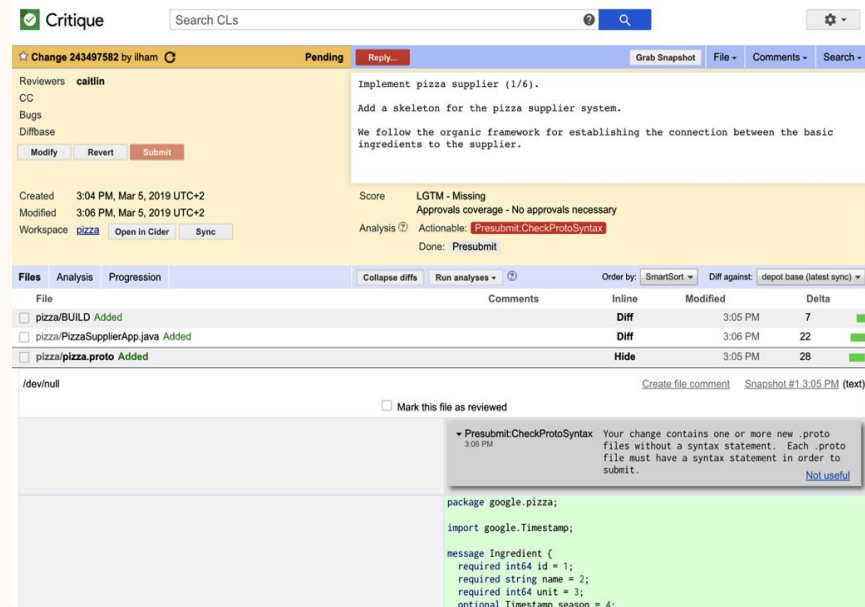


Figure 19-1. The code review flow

- Mandatory code review for all changes
- Millions of reviews yearly
- Focus on maintainability

What We Learn

- Maintainability improves
- Knowledge is shared
 - Faster onboarding
- Culture stays consistent
- Long-term quality ensured





Design Patterns



Adapter Design Pattern

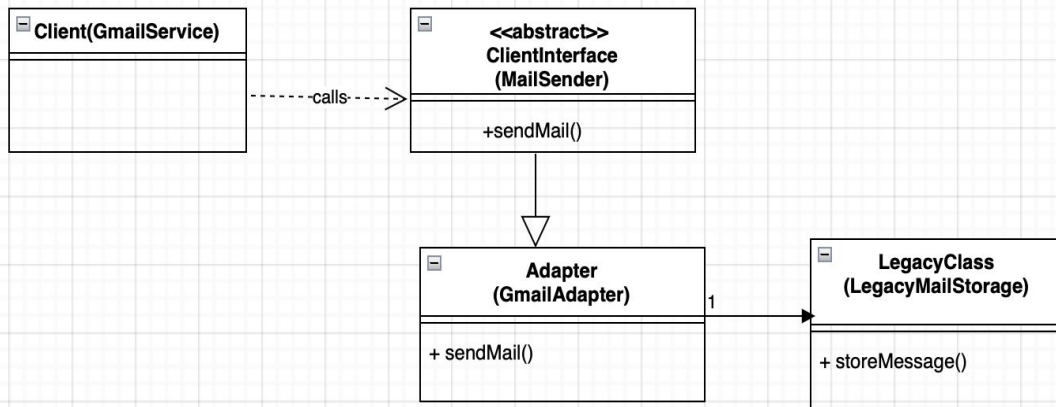
- Connects new APIs with 20-year-old legacy systems
- Avoids breaking existing infrastructure
- Common in Search, Ads, Gmail

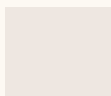
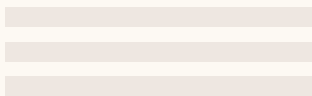
What It Is

A translation layer between new and old components

 Gmail storage upgrade via API adapter

ADAPTER DESIGN PATTERN AT GOOGLE





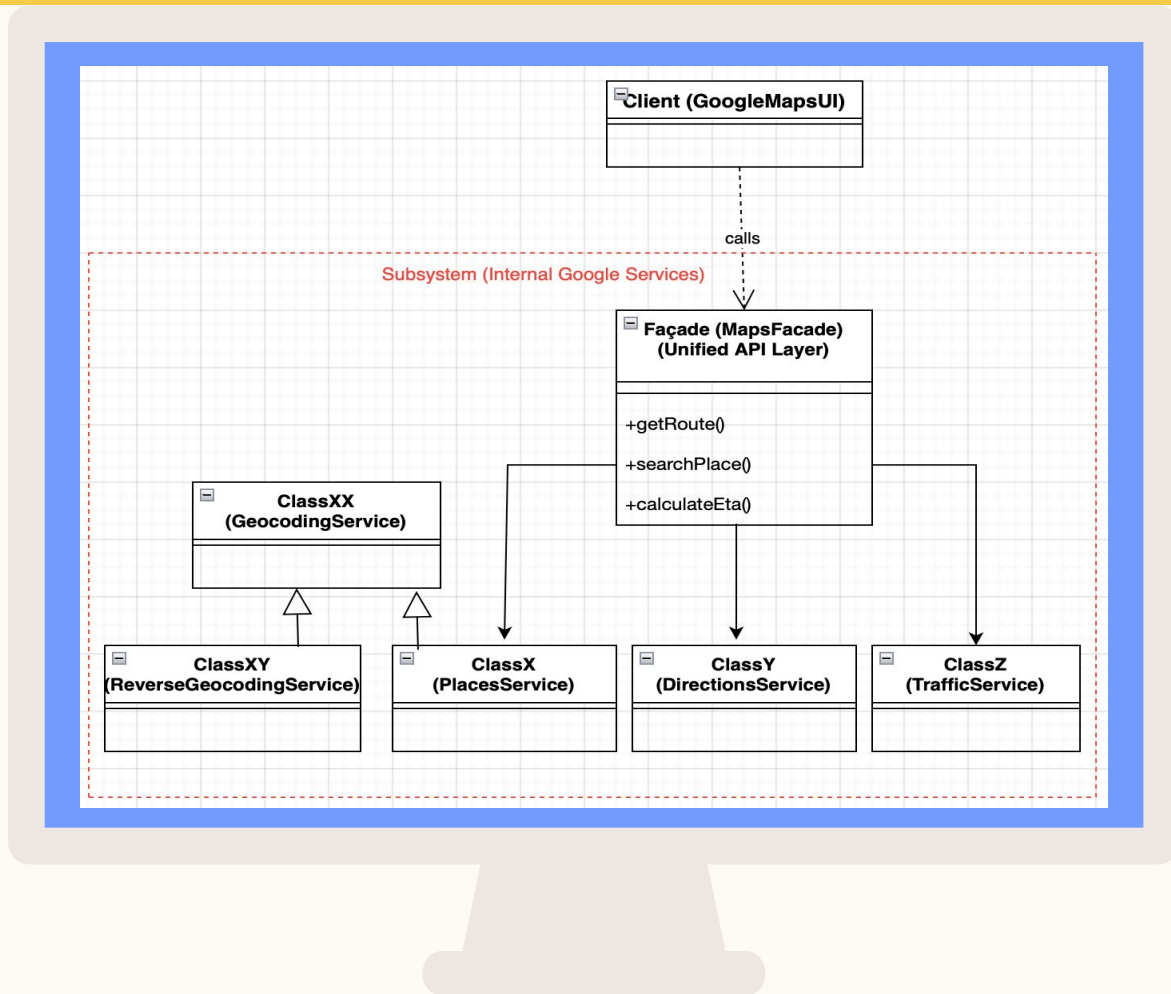
Façade Design Pattern

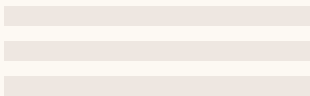
- Unifies multiple complex subsystems
- Prevents client code from depending on internal services
- Used in Maps, Firebase, Google Cloud APIs

What It Is

A simple interface that hides the complexity of many internal components.

Google Maps: unified API for routes, search, geocoding, traffic.





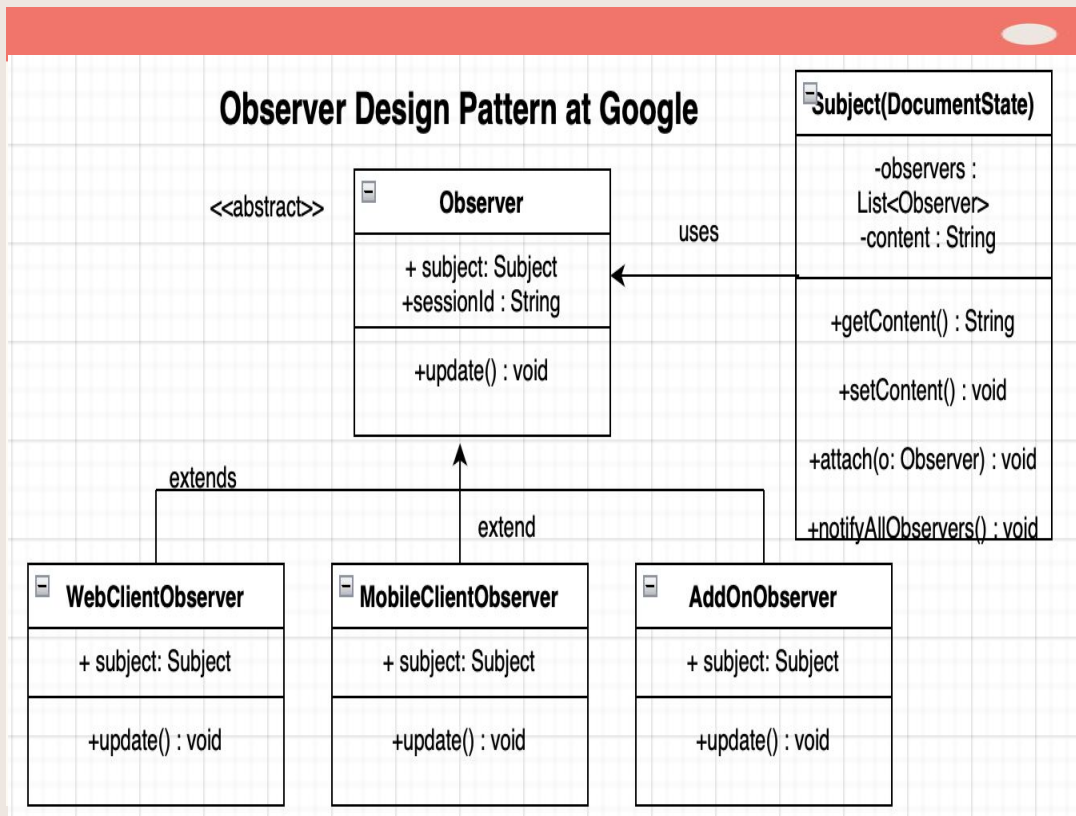
Observer Design Pattern

- Enables real-time synchronization across multiple clients
- Decouples data (Subject) from UI or external tools (Observers)
- Used in Google Docs, Drive, Sheets real-time collaboration

What It Is

A mechanism where changes in one object (the Subject) automatically notify all subscribed observers.

Google Docs: when a document changes, all connected clients (web, mobile, add-ons) instantly receive updates





CONCLUSIONS

Conclusion 1:

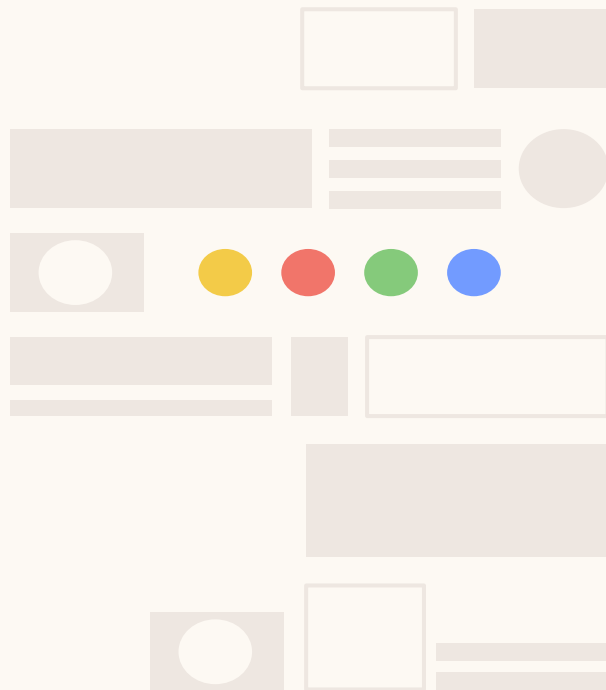
Google's long-term focus (maintainability, readability, discipline) keeps its massive codebase healthy.

Conclusion 2:

Design patterns like Adapter, Façade, and Observer ensure smooth integration, simplicity, and real-time collaboration.

Conclusion 3:

Consistent engineering discipline ensures Google's tools remain reliable and easy to evolve.



Thanks!

